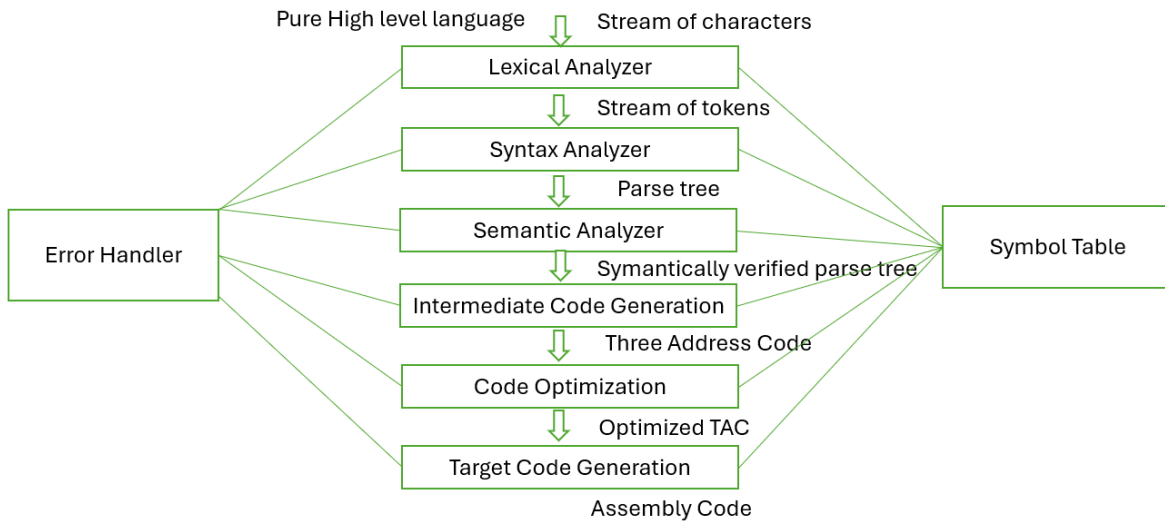
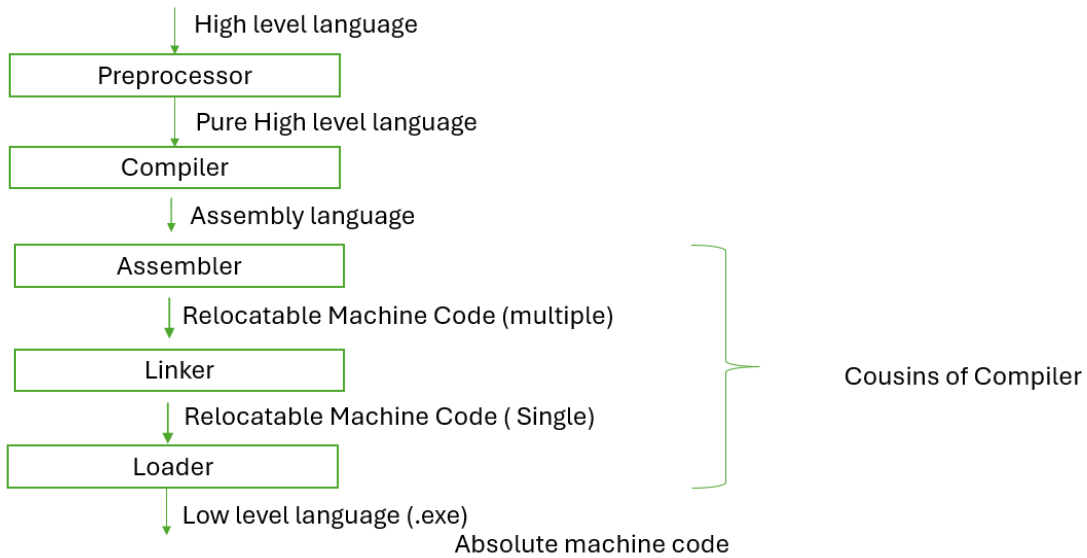


6 Phases of Compiler



LPS (Language Processing System)



How to check given Grammar is LL(1)

- The parsing table of a grammar may contain more than one production rule i.e. multiple defined. This is possible if G is left recursive or ambiguous.
- In this case, we say that it is not a LL(1) grammar.
- A grammar whose parsing table has no multiply-defined entries is said to be LL(1) grammar.

How to check given Grammar is LL(1)

- If G doesn't contain ε

$$A \rightarrow \alpha_1 \mid \alpha_2 \mid \alpha_3$$

$$\text{First}(\alpha_1) \cap \text{First}(\alpha_2) = \emptyset$$

$$\text{First}(\alpha_2) \cap \text{First}(\alpha_3) = \emptyset$$

$$\text{First}(\alpha_3) \cap \text{First}(\alpha_1) = \emptyset$$

- If G contains ε

$$A \rightarrow \alpha_1 \mid \alpha_2 \mid \varepsilon$$

$$\text{First}(\alpha_1) \cap \text{First}(\alpha_2) = \emptyset$$

$$\text{First}(\alpha_1) \cap \text{Follow}(A) = \emptyset$$

$$\text{First}(\alpha_2) \cap \text{Follow}(A) = \emptyset$$

LL(1) Parser – Parser Actions

- The symbol at the top of the stack (say X) and the current symbol in the input string (say a) determine the parser action.
- There are four possible parser actions.
 1. If ($X = \$$, $a = \$$) parser halts (successful completion)
 2. If $X = a$ (but not $\$$) parser pops X from the stack, and moves the next symbol in the input buffer.
 3. If X is a non-terminal
 - Parser looks at the parsing table entry $M[X, a]$.
 - If $M[X, a]$ holds a production rule $X \rightarrow Y_1 Y_2 \dots Y_k$, it pops X from the stack and pushes $Y_k, Y_{k-1}, \dots, Y_1$ into the stack.
 - The parser also outputs the production rule $X \rightarrow Y_1 Y_2 \dots Y_k$ to represent a step of the derivation.

LL(1) Parser – Parser Actions

4. None of the above : error
 - All empty entries in the parsing table are errors.
 - If X is a terminal symbol different from a , this is also an error case.

Ex2:

- $$\begin{aligned} S &\rightarrow aSA \mid \varepsilon \\ A &\rightarrow c \mid \varepsilon \end{aligned}$$

Check if this grammar is LL(1) or not

Solution:

- For this grammar $S \rightarrow aSA \mid \varepsilon$
 - $\text{First}(aSA) = \{a\}$, $\text{Follow}(S) = \{\$, c\}$, $\text{First}(aSA) \cap \text{Follow}(S) = \emptyset$
- For this grammar $A \rightarrow c \mid \varepsilon$
 - $\text{First}(c) = \{c\}$, $\text{Follow}(A) = \{\$, c\}$, $\text{First}(c) \cap \text{Follow}(A) = \{c\}$
- So, this grammar is not LL(1)

EX3:

- $$S \rightarrow aSbS \mid bSaS \mid \varepsilon$$

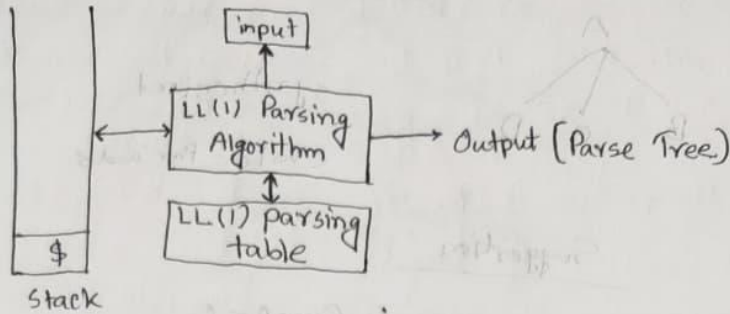
Check if this grammar is LL(1) or not

Solution:

For this grammar $S \rightarrow aSbS \mid bSaS \mid \varepsilon$

- $\text{First}(aSbS) = \{a\}$, $\text{First}(bSaS) = \{b\}$, $\text{Follow}(S) = \{\$, b, a\}$
- $\text{First}(aSbS) \cap \text{First}(bSaS) = \emptyset$
- $\text{First}(aSbS) \cap \text{Follow}(S) = \{a\}$
- $\text{First}(bSaS) \cap \text{Follow}(S) = \{b\}$
- So, this grammar is not LL(1)

LL(1) Parser



- ① No ambiguity
- ② no left recursion
- ③ First & Follow
- ④ Parsing Table
- ⑤ Stack Implementation
- ⑥ Parse Tree.

First and follow:

Grammar	First	Follow
E	{ id, (}	{ \$,) }
E'	{ E, + }	{ \$,) }
T	{ id, (}	{ +, \$,) }
T'	{ E, * }	{ +, \$,) }
F	{ id, (}	{ *, +, \$,) }

Ex: $E \rightarrow TE'$
 $E' \rightarrow E \mid +TE'$
 $T \rightarrow FT'$
 $T' \rightarrow E \mid *FT'$
 $F \rightarrow id \mid (E)$

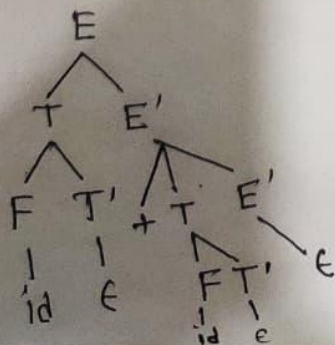
Parsing Table:

M	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow E$	$E' \rightarrow E$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow E$	$T' \rightarrow *FT'$		$T' \rightarrow E$	$T' \rightarrow E$
F	$F \rightarrow id$			$F \rightarrow id$		

Stack Implementation :

Stack	input	Output
\$ E	id + id \$	$E \rightarrow TE'$
\$ E' T	id + id \$	$T \rightarrow FT'$
\$ E' T' F	id + id \$	$F \rightarrow id$
\$ E' T' id	id + id \$	pop
\$ E' T'	+ id \$	$T' \rightarrow \epsilon$
\$ E'	+ id \$	$E' \rightarrow + TE'$
\$ E' T +	+ id \$	pop
\$ E' T	id \$	$T \rightarrow FT'$
\$ E' T' F	id \$	$F \rightarrow id$
\$ E' T' id	id \$	pop
\$ E' T'	id \$	$T' \rightarrow \epsilon$
\$ E'	id \$	$E' \rightarrow \epsilon$
\$	\$	accept.

Parse Tree



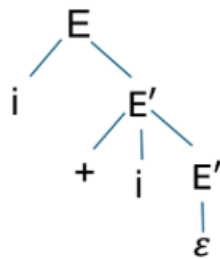
Recursive Descent Parser

- It is a top down parser.
- Non-deterministic grammar, Left recursive grammar and ambiguous grammar is not allowed here.
- Right recursive grammar is allowed here.

How Recursive Descent Parser works?

$E \rightarrow i E', E' \rightarrow + i E' \mid \varepsilon$

Let's take a string $i+i$.



We will write functions for corresponding variables. We will use $i+i\$$ ($\$$ represents the end of input or the end of a string)

How Recursive Descent Parser works?

```

E'()
{
    if(input=='+')
        { input++;
          if(input=='i')
              { input++;
                E'(); }
          }
    else
        {return;}
}
    
```

```

E ()
{
    If (input=='i')
        {input++;
          E'();}
    }
    
```

```

Main()
{
    E();
    if( input == '$')
        {printf("Parsing
          successful");}
    }
    
```

Recursion

$A \rightarrow A\alpha \mid \beta$

Eliminating left recursion

$A \rightarrow \beta A'$

$A' \rightarrow \epsilon \mid \alpha A'$

It is like a formula

EX1

$E \rightarrow E+T \mid T$

Now

$E \rightarrow E+T \mid T$

 $A \quad A \quad \alpha \mid \beta$

eliminating left recursion:

$E \rightarrow T E'$

$E' \rightarrow \epsilon \mid +TE'$

Recursion

$$A \rightarrow A\alpha \mid \beta$$

Eliminating left recursion

$$A \rightarrow \beta A'$$

$$A' \rightarrow \epsilon \mid \alpha A'$$

It is like a formula

Ex2:

$$S \rightarrow S01S0S \mid 01$$

Now,

$$\begin{array}{c} \boxed{S} \rightarrow \boxed{S} \boxed{01S0S} \mid \boxed{01} \\ \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \\ A \quad A \quad \alpha \quad | \quad \beta \end{array}$$

eliminating left recursion:

$$S \rightarrow 01S'$$

$$S' \rightarrow \epsilon \mid 01S0S S'$$

Left Factoring

- Converting nondeterministic grammar to deterministic grammar is called left factoring.

ND =>

Det

Non-Deterministic Grammar

- Grammar with common prefix between at least two different production from the same L.H.S (non-terminal in CFG) is called non-deterministic grammar.

$$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \mid \alpha\beta_3 \mid \alpha\beta_4$$

Suppose you have to generate $\alpha\beta_4$

- Such grammar requires backtracking due to confusion which is actually a costly process.
- After tracing th first input ' α ', there are 4 available options. So the confusion is to go with $\beta_1, \beta_2, \beta_3, \beta_4$
- Due to this confusion, Top down parser don't prefer to parse Non-determinestic gramar.

Checking Non-deterministic Grammar

- **EX1:** Check if the grammar is non-deterministic or not

$S \rightarrow aSb \mid aA \mid b$

$A \rightarrow aB \mid a$

$B \rightarrow b$

Ans: Yes.

- **EX2:** Check if the grammar is non-deterministic or not

$S \rightarrow aSb \mid bSaA \mid \epsilon$

Ans: No

- **EX3:** Check if the grammar is non-deterministic or not

$S \rightarrow Aab \mid BA$

$A \rightarrow a \mid b$

$B \rightarrow d \mid e$

Ans: No.

- **Ex4:** Check if the grammar is non-deterministic or not

$A \rightarrow a\beta_1$

$B \rightarrow a\beta_2$

Ans: No.

3/3/25

20

Deterministic grammar

- Without any common prefix in any of the different productions from same L.H.S (non-terminal in CFG).
- Examples:
 - EX2, EX3, EX4

Left factoring

Formulla

$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \mid \alpha\beta_3 \mid \alpha\beta_4$

Now converting it to deterministic

$A \rightarrow \alpha A'$

$A' \rightarrow \beta_1 \mid \beta_2 \mid \beta_3 \mid \beta_4$

• EX1:

$S \rightarrow iEtS \mid iEtSeS \mid a$

$E \rightarrow b$

Convert it to deterministic grammar

Solution:

$S \rightarrow iEtSS' \mid a$

$S' \rightarrow \epsilon \mid eS$

$E \rightarrow b$

3/3/25

22

Left factoring

- **EX2:**

$S \rightarrow aSSbS \mid aSaSb \mid abb \mid b$

Convert it to deterministic grammar

Solution:

$S \rightarrow aS' \mid b$

$S' \rightarrow SSbS \mid SaSb \mid bb$

$S' \rightarrow SS'' \mid bb$

$S'' \rightarrow SbS \mid aSb$

- So the final result is:

$S \rightarrow aS' \mid b$

$S' \rightarrow SS'' \mid bb$

$S'' \rightarrow SbS \mid aSb$

Left factoring

- **EX4:**

$S \rightarrow a \mid ab \mid abc \mid abcd$

Convert it to deterministic grammar

Solution:

$S \rightarrow aS'$

$S' \rightarrow \epsilon \mid b \mid bc \mid bcd$

$S' \rightarrow \epsilon \mid bS''$

$S'' \rightarrow \epsilon \mid c \mid cd$

- $S'' \rightarrow \epsilon \mid cS'''$
 $S''' \rightarrow \epsilon \mid d$

So, the final result is

$S \rightarrow aS'$

$S' \rightarrow \epsilon \mid bS''$

$S'' \rightarrow \epsilon \mid cS'''$

$S''' \rightarrow \epsilon \mid d$

TOP DOWN PARSER VS BOTTOM UP PARSER

Aspect	Top-Down Parser	Bottom-Up Parser
Starting Point	Begins parsing from the start symbol (root) .	Begins parsing from the input tokens (leaves) .
Parsing Direction	Expands productions from top to bottom of the parse tree.	Reduces input from bottom to top into the start symbol.
Approach	Prediction-based : chooses which production to apply.	Reduction-based : combines tokens into higher-level rules.
Builds Tree	Builds the parse tree from root to leaves .	Builds the parse tree from leaves to root .
Common Techniques	Recursive Descent, LL Parsers	Shift-Reduce, LR Parsers
Backtracking	Often needs backtracking (unless LL(1))	Generally no backtracking needed
Ease of Implementation	Easier to write manually	More complex; usually generated by parser tools
Efficiency	Less efficient for ambiguous grammars	More efficient and powerful; handles a wider class of grammars
Grammar Type Supported	Mostly works for LL grammars	Works for LR grammars (more general)

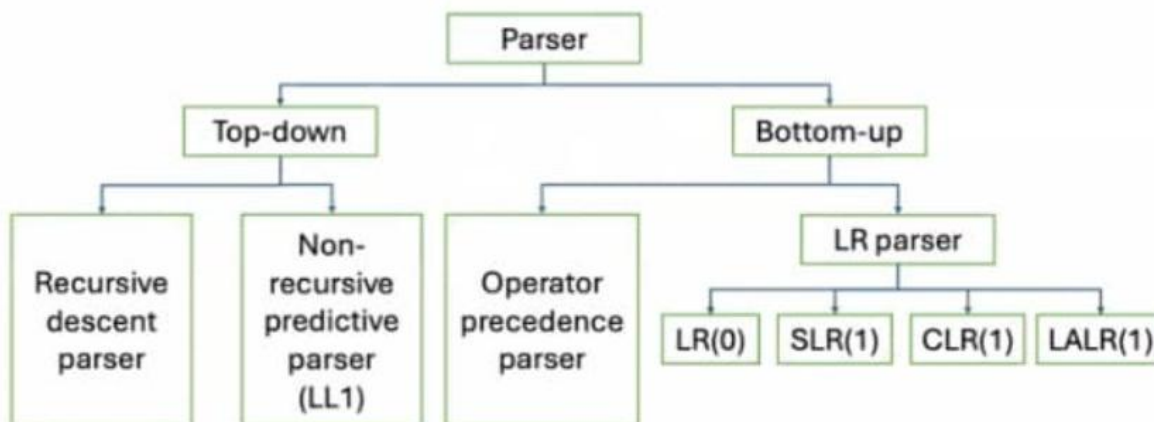
Regular Grammar vs Context-free Grammar

Regular Grammar	Context-free Grammar
It is not suitable for parsing.	Parsing is suitable
Type-3 Grammar	Type-2 Grammar
It is a subset of CFG	Every CFG may not be RG.
Syntax and any programming language can't be represented completely by RG	Syntax and any programming language can be represented completely by CFG

Types of SDD

S - Attributed	L - Attributed
A SDD that uses only synthesized attribute is called S-Attributed SDD. Ex: $A \rightarrow BCD$ $A.i = B.i$ $A.i = C.i$ $A.i = D.i$	A SDD that uses both synthesized and inherited attributes is called as L-attributed SDD but each inherited attribute is restricted to inherit from parent or left siblings only. Ex: $A \rightarrow BCD$ $C.i = A.i$ $C.i = B.i$ $C.i = D.i \text{ (Not allowed)}$
Semantic actions are always placed at right end of the production (Postfix SDD)	Semantic action are placed anywhere on the R.H.S of the production
Use bottom up parsing	Use top down parsing
It is a subset of L attribute	Every L attribute is not a S attribute

Types of Parser



Check The Type of Attributes

- $A \rightarrow BC \{ B.i = A.i, C.i = B.i, A.i = C.i \}$
 - Ans: L attribute, Not S attribute
- $A \rightarrow QR \{ R.i = A.i, Q.i = R.i, A.i = Q.i \}$
 - Ans: Not L attribute, Not S attribute
- $A \rightarrow PQ \{ A.i = P.i, A.i = Q.i \}$
 - Ans: L attribute, S attribute

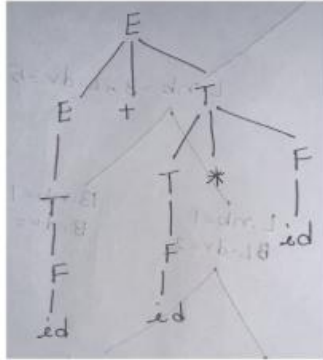
SDD to Syntax Tree

Production	Semantic Rules
$E \rightarrow E + T$	$\{ E.nptr = \text{mknode}(E.nptr, +, T.nptr) \}$
$E \rightarrow T$	$\{ E.nptr = T.nptr \}$
$T \rightarrow T * F$	$\{ T.nptr = \text{mknode}(T.nptr, *, F.nptr) \}$
$T \rightarrow F$	$\{ T.nptr = F.nptr \}$
$F \rightarrow \text{id}$	$\{ F.nptr = \text{mknode}(\text{Null}, \text{id}, \text{Null}) \}$

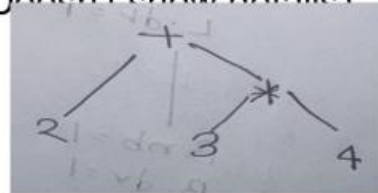
- In order to understand syntax tree let's derive a parse tree for this string '2+3*4'.

SDD to Syntax Tree

- This is parse tree. It's another name is concrete syntax tree.

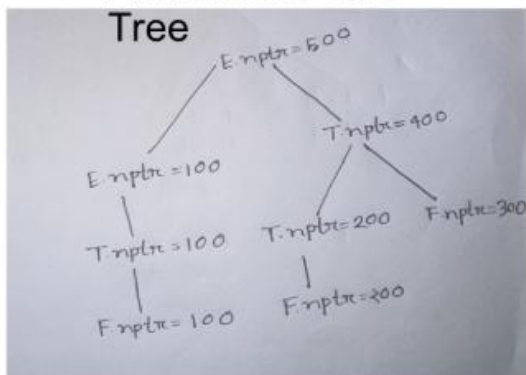


- We can use less no of nodes and represent the same string.
- Then it will be called syntax tree or abstract syntax tree. (Doesn't show details)

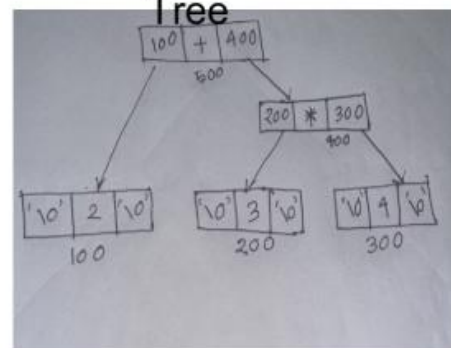


SDD to Syntax Tree

Annotated Parse Tree



Syntax Tree



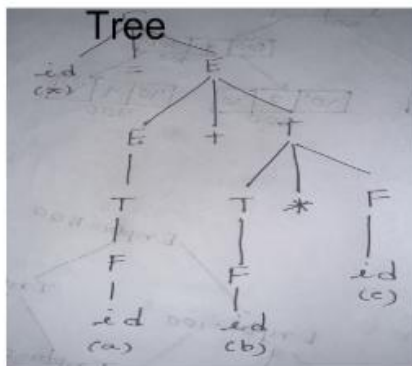
SDD to Generate Three Address Code

Production	Semantic Rules
$S \rightarrow id = E$	$\{ gen(id.name = E.place); \}$
$E \rightarrow E + T$	$\{ E.place = new\ temp();$ $gen(E.place = E.place + T.place); \}$
$E \rightarrow T$	$\{ E.place = T.place \}$
$T \rightarrow T * F$	$\{ T.place = new\ temp();$ $gen(T.place = T.place * F.place); \}$
$T \rightarrow F$	$\{ T.place = F.place \}$
$F \rightarrow id$	$\{ F.place = id.name \}$

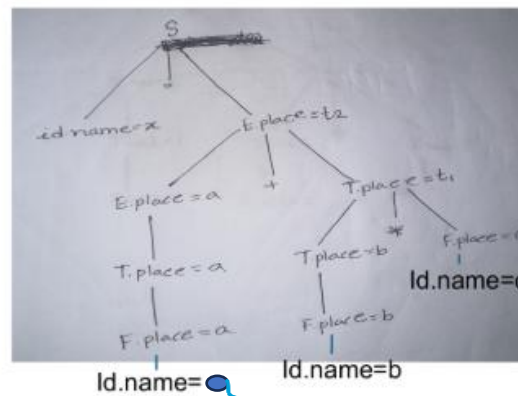
SDD to Generate Three Address Code

- In order to understand syntax tree let's derive a parse tree for this string 'x=a+b*c'.

Parse



Annotated Parse



Output

$t_1 = b * c$
 $t_2 = a + t_1$
 $x = t_2$

Introduction

- SDD = Grammar + Semantic Rules
- Semantic means the parse tree which is generated is meaningful.
- Meaningful means the operations which are performed maintained the rules.
- Applications:
 - Executing arithmetic expressions
 - Conversion from binary to decimal
 - Creating syntax tree
 - Type checking etc.

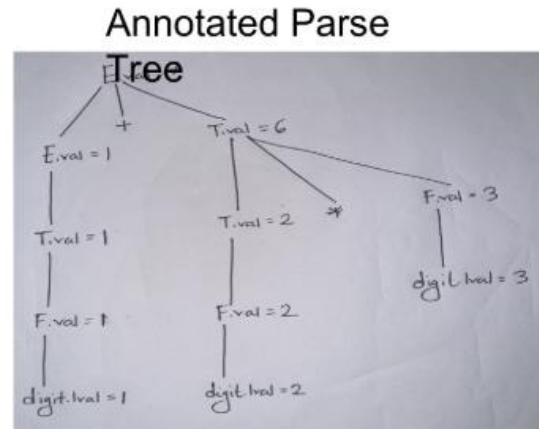
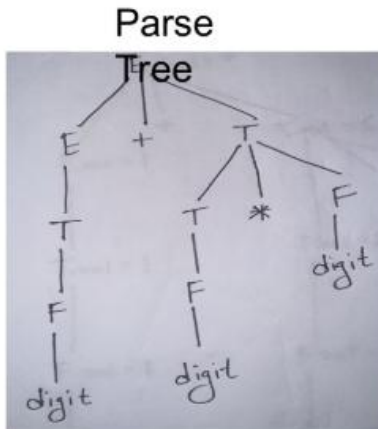
Productions and Semantic Actions (Example1)

Production	Semantic Rules
$E \rightarrow E + T$	{ $E.val = E.val + T.val$ }
$E \rightarrow T$	{ $E.val = T.val$ }
$T \rightarrow T * F$	{ $T.val = T.val * F.val$ }
$T \rightarrow F$	{ $T.val = F.val$ }
$F \rightarrow \text{digit}$	{ $F.val = \text{digit.lval}$ }

- Attributes are associated with grammar symbols and semantic rules are associated with productions.
- Attributes value may be number, strings, reference, datatype etc.

Example1 (Cont'd)

- Given the string $1 + 2 * 3$. Now by using annotated (Symanctically Verified) parse tree show the output



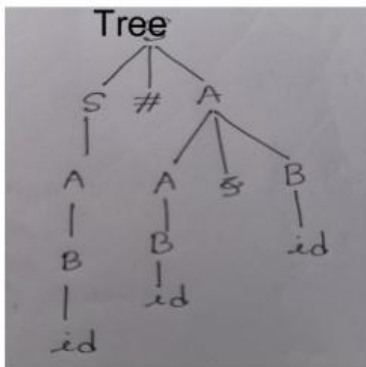
Example2

Production	Semantic Rules
$S \rightarrow S \# A \mid A$	$\{ S.val = S.val * A.val, S.val = A.val \}$
$A \rightarrow A \& B \mid B$	$\{ A.val = A.val + B.val, A.val = B.val \}$
$B \rightarrow id$	$\{ B.val = id.lval \}$

Given the string $5 \# 3 \& 4$, Now by using annotated parse tree show the output

Example2 (Cont'd)

Parse



Annotated Parse

